

2026 CSP-S 复赛赛前模拟赛 Day2 复盘

比赛：代码源新 OJ · 2026 CSP-S 复赛赛前模拟赛 day2 · OI 赛制 · 灵活时间 4 小时（你是北京时间 10:10 到 14:10） · 四题都是 1 秒 / 1024 MiB · 必须 freopen

结果：130 / 400（A 100，B 0 未交，C 30，D 0）。比 Day1 的 45 分多 85 分

分析依据：归档（题面 + 题目列表页快照，里面有你全部 6 次提交）、赛时录音转写（10:03 到 14:10）、虚拟机赛后桌面和 Downloads（每个代码版本、大样例、D 的 checker）。四题正解我都写了，用大样例和暴力对拍验证过。草稿纸按你说的没要。

零、一页结论

先说好消息：A 是你自己推出来的。连续 1 算两项、中间只能夹一个 0——这些全是你赛时推出的结论。最后 18 分钟你写了暴力、开了对拍，2 分钟就抓到反例，13:35 满分。这正是 Day1 没有的能力，要保住。

但这场还是丢了很多分，原因就三条：

- 1. 有了等式，没马上写暴力。11:07 你已经说出“把 A 拆成一个数减另一个数”（录音 1:04:09）。这句话本身就是暴力：枚举减掉的那个数。可是暴力 13:17 才写，晚了 2 小时 10 分。A 一共花了 2 小时 37 分，占全场 65%，比 Day1 的 55% 还高。
- 2. B 你离正解只差一句话。11:45 你说“每一层求个最大值”，11:49 说“插入一个串，这条路径上全加一，每层再取最大值”——这就是正解的前一半。你以为“字典树退化链会很慢”就放弃了，其实插入的总代价就是 $\sum |s| \leq 10^6$ 。B 的 10 分暴力也没写。
- 3. D 两次提交都是 0 分，而且都是能避免的。
 - 14:01 那版暴力：枚举狼的分布时，你按 `i` 的二进制位数取位，结果只试了“最后一个人是狼”的那一半情况。题面样例三组答案恰好都是“最后一个是狼”或无解，所以样例过了；大样例 `sample2`（正好是子任务 1 的 1000 组数据）一跑就会错 277 组，你没跑。
 - 14:09 那版：`solve2` 先用后声明，编译错误。离结束还有 14 秒，交之前没编译（可执行文件是 14:00 的，源码是 14:09 的）。

按你现在的水平，这场合理分数：

题	实际	稳拿线	努力线	差在哪
A	100	100	100	拿到了，但花了 65% 的时间
B	0	10	40	10 分暴力没写；11:49 的想法写出来就接近正解
C	30	30	30	拿到了。60 分以上要二维偏序，现阶段不强求
D	0	10	25	暴力少枚举一半；全是 L 的 15 分是两状态 DP
合计	130	150	195	

下场只改三件事：

1. 写出等式就写暴力：只要纸上出现 $a = P - N$ 、 $f = \max \dots$ 这种式子，5 分钟内把它变成暴力。暴力是打表机和对拍器，不是最后才写的骗分。
2. 交之前三件事：刚编译过、样例和大样例全过、交的就是测的那个文件。做不到就不交。
3. T+1:00 还没交过任何题，立刻停下写暴力交上。这场第一次提交是 T+2:35。

一、证据完整性检查

已看到：

- 归档：4 题题面、时空限制、子任务表；题目列表页快照里有你每题最终得分和 6 次提交的时间、状态、耗时
- 录音转写 1 份（北京 10:03 到 14:10，全程）
- 虚拟机赛后桌面 + Downloads：A 的 7 个版本、C 的 2 个版本、D 的 1 个版本、B 的开头；三题大样例；D 的 checker；对拍文件夹

没看到：

- 榜单和名次（OI 赛制，平台没给看）。所以本报告没有同分段对比
- 其他选手的代码
- 草稿纸（你说不用）；回收站（这次导出里没有 Day2 的回收站）

我纠正的数据：

项目	归档里写的	实际	怎么查出来的
本人提交	0 次，“完全未提交 4 题”	6 次，130 分	提交接口返回 403，但 01_比赛信息/题目列表服务器页面.html 里有提交表
D 的题名	“0 Compile Error”		归档脚本把状态栏当成了题名。“0 Compile Error”其实是你 D 题的最后状态
文件时间	—	桌面导出 ZIP 里的时间 +8 小时是北京时间	用 6 次提交时间逐一对齐（例：T4.cpp 06:09 ↔ D 提交 14:09:46）。不再是 Day1 的 +12
录音起点	转写文件头写 2026-09-26 22:03	北京 10:03:02	文件头是纽约时间；用 3 次提交对齐：录音 2:42:11 说“交上去了”↔ 12:45:13，3:03:33 ↔ 13:06:36，4:06:44 ↔ 14:09:46

下文时间都是北京时间；“T+2:35”指开赛后 2 小时 35 分（开赛 10:10）。

二、比赛总画像

题	名字	本质	你的状态	分
A	乘法	用最少数个 $\pm 2^k$ 凑出 a （有符号二进制），答案 = $2 \times \text{项数} - 1$	3 次提交：RE → WA → AC	100
B		字典树 + 每层最大计数，增量维护答案	想了 30 分钟，没交	0
C		前缀差 + 二维偏序 DP； $n = 2 \times 10^6$ 要利用“每一步两维同涨同跌”	$O(n^2)$ DP，TLE	30
D		村民链 + 总狼数只在“左看/右看交接处”被钉死	暴力 0 分，最后一交 CE	0

6 次提交（全部来自页面快照）：

时间	题	结果	交的是哪份
12:45:13	A	0 Runtime Error	problems/T1.cpp (12:22) , while 条件写成了 <code>\\ </code> , 数组越界
13:06:36	A	0 Wrong Answer	time.cpp (13:02) , 大样例输出 729, 你当时已经知道应该是 671
13:35:19	A	100 Accepted	duipai_T1/wa.cpp (13:33) , 加了 check 函数
13:51:49	C	30 Time Exceeded	debug/main.cpp (13:49) , $O(n^2)$
14:01:08	D	0 Time Exceeded	14:00 编译的暴力 (枚举漏一半)
14:09:46	D	0 Compile Error	problems/T4.cpp (14:09) , solve2 未声明

时间都花在哪了：

去向	时长	占比
A (10:10–11:39、12:13–12:45、12:59–13:35)	2 小时 37 分	65%
B (11:39–11:52、12:45–12:59、14:01–14:04)	约 30 分钟	12%
C (11:52–12:05、13:35–13:51)	约 29 分钟	12%
D (12:05–12:13、13:51–14:01、14:04–14:09)	约 23 分钟	10%

其中环境问题约 12 分钟：复制粘贴失灵 (10:44、12:38)、终端报 `word unexpected` (11:22–11:29 , 7 分钟)、Code::Blocks “抽风”后改用终端 (12:31) 。比 Day1 的 30 分钟少了一大半, “5 分钟修不好就回终端”这条规则起作用了。

一句话判断：想法质量在变好 (A 全靠自己推, B 想到了一半, C 想到了前缀差) , 但“想法 → 分数”这一段还是漏：暴力写得太晚、交的东西没测、四题读完用了近 2 小时。

三、时间线

北京时间	T+	发生了什么	评价
10:09	—	进比赛, 注意到“这次要文件输入输出”	✓
10:11	0:01	先把所有附件下载下来	✓ 好习惯
10:16	0:06	推出“答案 = $2 \times \text{项数} - 1$ ”	✓ 完全正确
10:17	0:07	想到“DFS 加记忆化”	✓ 方向对了, 后来被自己否掉
10:17–10:26		两种构造：直接按 1 拆 / 高一位的 2^n 减去补数, 取小的	思路太窄
10:27–10:46		写二进制高精度减法, 开断点调试	
10:48	0:38	大样例 $947 \neq 671$	该触发止损
10:49	0:39	“为什么一定要在最高位呢?”	✓ 发现问题
10:59	0:49	“跑个记忆化, 记录所有出现过的……位置太多了, 算了”	✗ 关键误判：状态只需要“第几位 + 进位”, 一共 $2n$ 个

北京时间	T+	发生了什么	评价
11:07	0:57	“只会用一次加法和一次减法……把 A 拆成 A 减 B”	✓✓ 关键转化，这句话就是暴力
11:10– 11:19		“要最小化加的位数和减的位数之和”，然后“两个是连锁的，卡住了”	该写暴力打表
11:22– 11:30		写程序数样例里 1 的个数（474 vs 671）；中间卡在 word unexpected 7 分钟	✓ 想打表的方向对，但只打了一个数
11:33	1:23	“几个零之间肯定只隔一个一”	✓ 这就是答案的结构（非相邻形式）
11:39	1:29	转 B	T+1:29 才开第二题
11:45– 11:49		字典树；“每一层求个最大值”；“插入时路径上全加一，每层取最大”	✓✓ 正解前半
11:50	1:40	“字典树退化成链……复杂度差得远”	✗ 复杂度估错（见四.B）
11:52– 11:58		读 C，写出 $O(n^2)$ DP	✓
11:58– 12:05		想优化：“只要知道 A-B、A-C”，“i 减去 f[i]”	✓ 方向全对
12:05	1:55	“N 方的分非常好骗……30 分手到擒来”，但没写	✗ 违反“先交暴力”
12:07– 12:13		读 D，看 checker	✓ 执行卡“先看 checker”做到了
12:13	2:03	回 A	
12:18	2:08	“我会了，发现诀窍了”	连续 1 算两项——对了一大半
12:22		写完 v2，while (point <= n \\ \\ a[point] != 1)	✗ 越界
12:28		大样例 $729 \neq 671$	该止损
12:33		改系数试出 853	✗ 碰运气
12:38		发现“Linux 复制的窗口关了就粘不了”	✓ 记下来了
12:45:13	2:35	第一次提交，A → RE（交的是 12:22 那份）	✗ 交的不是测过的
12:45– 12:59		想 B、算分：“40 + 30 + 10 + 15……”	✓ 在算分，但没行动
13:06:36	2:56	重交 A → WA（交的是已知输出 729 的版本）	✗ 过不了样例还交
13:07		“不是代码问题，是思路有问题。但我认为我的思路没有问题。”	下一步做对了：去写暴力
13:10		“我会写暴力吗？我最近暴力咋写？”	
13:17	3:07	“等一下，有可能是生成两个”——想到枚举 b	✓ 这就是 11:07 那句话
13:21		“1000 组没问题”，然后“一个数据都没生成呢”	✗ gen.cpp 是空模板
13:22		真开拍，第一组就出错	✓
13:24– 13:28		手算反例：“中间只能接受一个 0”	✓ 最后一块拼图
13:35:19	3:25	A → 100	✓

北京时间	T+	发生了什么	评价
13:36–13:48		写 C (写进了 T2.cpp)，发现“大于等于我搞错了”	✓ 样例抓住了读题错
13:51:49	3:41	C → 30	✓
13:52–14:00		D 暴力，直接搬 checker 的 isValid	✓ 聪明
14:01:08	3:51	D → 0 (TLE)	✗ 没跑大样例
14:01–14:04		B 在 T3.cpp 里开了头，“字典树难写，性价比不高”	
14:04–14:09		D 子任务 2：b[i-1] > b[i] 就输出 1	✗ 思路也不对
14:09:46	3:59	D → CE	✗ 最后 14 秒交没编译的代码

四、逐题复盘

A 乘法 (100 分，用时 2 小时 37 分)

题目本质：把 a 写成若干个 $\pm 2^k$ 的和，项数最少是多少（每项一次左移，项与项之间一次加减，所以答案 = $2 \times \text{项数} - 1$ ）。最少项数就是 a 的“非相邻形式”（NAF）里非零位的个数。

你的思路复原（每一步都是你自己推的）：

- 10:16 答案 = $2 \times \text{项数} - 1$ ✓
- 10:17 两种构造：按 1 拆 / 高一位减补数，取小 → 大样例 $947 \neq 671$
- 11:07 $a = P - N$ ：加的项合起来是 P ，减的项合起来是 N ，项数 = P 的 1 的个数 + N 的 1 的个数 ✓✓
- 11:33 “几个零之间只隔一个一” ✓
- 12:18 连续一段 1 用“高一位减最低位”两项搞定，单独的 1 一项 ✓ (v4，大样例 729)
- 13:28 一段 1 中间夹一个 0 时，多花一项就能把后面接上；夹两个 0 就断开 ✓ (v5，AC)

你的 13:06 版本 (v4) 为什么错——最小反例 **11011** (= 27)：

- v4：11 一段两项，11 又一段两项，共 4 项，输出 7
- 实际：27 = 32 - 4 - 1，3 项，答案 5。中间那个单独的 0 让两段可以合并

我把 12 位以内的所有 a (4095 个) 都和暴力对过：v4 错 1085 个，v5 (你最后交的) 一个不错。

你偏掉的两个地方：

时刻	你说的	问题	应该怎样
10:59	“记录所有出现过的……位置太多了，算了”	你想要的状态是“整个数”，当然太多	状态只要“做到第几位 + 有没有进位”，一共 $2n$ 个
11:07 → 13:17	“把 A 拆成 A 减 B”	等式有了，暴力就有了：枚举 B	11:10 就能写出暴力，打表看规律

不需要任何灵感的正解：数位 DP (你 10:17 想的“记忆化”就是它)

从最低位往高位做，每一位最后填 -1、0、+1 中的一个，状态只记“往上一位进位 0 还是 1”：

当前位 + 进位	可以怎么填	新进位	项数
0	填 0	0	不变
1	填 +1	0	+1
1	填 -1（相当于借了一个 2）	1	+1
2	填 0	1	不变

最后如果还有进位，再加一项。15 行，见 正解代码/A_乘法.cpp。这是这类题的通用解法：二进制、加减、“最少用几项”，就从低位做，状态记进位。

如果 11:10 写了暴力会怎样：暴力 A_暴力.cpp 12 行（你 13:17 写的那份几乎一样）。打出 1~64 的答案，“中间一个 0 可以吞、两个 0 断开”一眼就能看出来。按你 13:17 → 13:35 的速度，A 在 11:40 前后就能过，多出来近 2 小时给 B 和 D。

下次识别信号：

- 二进制 + 加减 + 最少项数 → 从低位做，状态 = 进位
- 纸上出现“A = X - Y”这种等式 → 枚举一边，就是暴力

专项：LeetCode 2571《将整数减少到零需要的最少操作数》（同一题，数据小），用数位 DP 写一遍，再用贪心写一遍。

B (0 分，未交)

题目本质：对每个 i ， $f(i, j)$ = 前 i 个串里选 j 个的最长公共前缀，求 $\sum_j f(i, j) \text{ xor } j$ 。

你的思路复原：

- 11:41 字典树，选 j 个串就是找一个结点，它下面至少有 j 个串 ✓
- 11:42 想二分深度
- 11:45 “我给每一层求个最大值” ✓✓
- 11:49 “插入一个串，这条路径上全加一，然后每一层都要取一遍最大值” ✓✓
- 11:50 “如果字典树退化成链……难办了……复杂度差得远” ✗
- 12:45 “你是怎么做到不挨个算就能知道的？”

你已经有的：设 $mx[d]$ = 深度 d 上经过次数最多的结点的次数。那么 $f(i, j)$ = 最大的 d ，使得 $mx[d] \geq j$ 。这就是你 11:45 说的“每层最大值”。

你卡住的第一处——复杂度估错了：插入第 i 个串只会经过 $|s_i|$ 个结点，每个结点 +1，每层最多一个结点变。所有串加起来 $\sum |s_i| \leq 10^6$ 。退化成链也一样：链长就是串长。估复杂度要按“总和”算，不要按“最坏一次 × 次数”算。

你卡住的第二处——“不挨个算怎么知道”：

- 插入一个串时，每层的 $mx[d]$ 最多 +1

- `mx[d]` 从 m 变成 $m+1$ ，只影响一个 f ： $f(m+1) = \max(f(m+1), d)$ 。因为 $f(j)$ 的定义里只有 $j = m+1$ 这一个条件变了
- 所以答案可以增量维护：改之前减掉 `f xor j`，改之后加回来
- 每个 i 还要新加 $j = i$ 这一项，它一开始是 `0 xor i`

一共改 $\sum |s_i|$ 次， $O(\sum |s_i|)$ 。核心就这几行（完整代码 正解代码/B_毛线.cpp）：

```
ans += (0 ^ i);
for (int d = 0; d <= len; d++) {
    // u 走到第 d 层的结点
    cnt[u]++;
    if (cnt[u] > mx[d]) {
        mx[d] = cnt[u];
        int j = mx[d];
        if (d > f[j]) {
            ans -= (f[j] ^ j);
            f[j] = d;
            ans += (f[j] ^ j);
        }
    }
}
```

大样例 5 个全过，最大的 0.25 秒。

稳拿的 10 分：子任务 1 ($n \leq 10$, $|s| \leq 50$) 按定义暴力，正解代码/暴力/B_暴力.cpp 30 行。你 14:01 说“性价比不高”——10 分钟 10 分，比 D 那 5 分钟写错的 solve2 性价比高得多。

下次识别信号：

- “对每个前缀 i 都要输出答案” → 想增量：加一个元素，哪些量会变、变多少
- “ $f(j)$ = 最大的 d 使得 $g(d) \geq j$ ” → f 和 g 互为“转置”， g 的一次 $+1$ 只改一个 f

专项：字典树模板题练手（洛谷 P8306）；然后自己把 B 从空白写一遍。

C (30 分, $O(n^2)$)

题目本质：设 X = 前缀的（红 - 黄）， Y = 前缀的（红 - 蓝）。段 (j, i) 是好的 $\Leftrightarrow X[j] \leq X[i]$ 且 $Y[j] \leq Y[i]$ 。

DP： $f[i] = \max(f[i-1], \max \text{ 好的 } j \text{ 的 } f[j] + i - j)$ 。

你的思路复原：

- 11:55 “要知道上一次切割点” → 11:58 写出 $O(n^2)$ DP ✓
- 12:01 “只要知道 $A - B$ 的值”“ $A - C$ ” ✓（就是前缀差 X 、 Y ）
- 12:04 “如果我以…… i 减去 $f[i]$ ” ✓（正解里的关键量 $u = i - f[i]$ ）
- 12:05 “30 分手到擒来”，但 13:36 才开始写
- 13:40 第一版写成了严格大于，样例 1 输出 1 → 13:48 “大于等于我搞错了” ✓

做对的：DP 定义、前缀差、`i - f[i]` 这个量都想到了；样例抓住了“大于等于”。

做错的：12:05 想到 30 分却推迟了 1.5 小时才写。按执行卡“先交暴力”，12:05 就该花 10 分钟交上。

为什么 2×10^5 和 2×10^6 有区别（你 12:05 问的）： 2×10^5 允许 $O(n \log^2 n)$ （二维偏序，CDQ 分治），可以拿 60 分； 2×10^6 在 1 秒内只能 $O(n)$ 或很轻的 $O(n \log n)$ ，必须找题目的特殊结构。

正解思路（供了解，现阶段拿 30 分就是对的）：

1. 设 $u[i] = i - f[i]$ ，意思是“前 i 格里最少有几格不在好段里”。转移： $u[i] = \min(u[i-1] + 1, \text{所有 } X[j] \leq X[i] \text{ 且 } Y[j] \leq Y[i] \text{ 的 } j \text{ 的 } u[j])$
2. 特殊结构：有红子的格子， X 、 Y 都不减；没有红子的格子， X 、 Y 都不增、且至少一个减。点 (X, Y) 每一步要么往右上走、要么往左下走，两维永远同方向
3. 往右上走时，新“看得到”的点只在一列和一行上 → 每列、每行各维护一个有序表
4. 往左下走时， u 只可能不变或 $+1$ → 只要判断“有没有 u 值等于当前 u 的点在左下方” → 每个 u 值维护一条阶梯（极小点集合）
5. 只有“没红子而且没被覆盖”的格子才需要记下来

$O(n \log n)$ ， 2×10^6 各种分布最慢约 0.4 秒。和 $O(n^2)$ 暴力随机对拍 900 组全对。代码 正解代码/C_棋子.cpp。

下次识别信号：“段内 $A \geq B$ 且 $A \geq C$ ” → 前缀差 + 二维偏序； n 到百万级 → 一定有特殊结构，先看每一步能怎么变。

专项（有余力再做）：一维版本“段和 ≥ 0 ，总长度最长”用树状数组做；二维偏序入门洛谷 P3810。

D 🐾 (0 分，TLE 然后 CE)

题目本质：村民排成一条链，相邻两个村民之间最多夹一个狼。左看的村民定死了“前面有几个狼”，右看的村民定死了“后面有几个狼”。

你的思路复原：

- 12:08 看 checker ✓（执行卡第四节的规则，这次做到了）
- 12:52 子任务 2：“第一个人肯定说 0，不是 0 直接拖出去；比它多 1 的话左边那个是狼” ✓ 方向对
- 12:57 “列出所有情况，总共 8 种……太复杂了，只能拿个十分”
- 13:53 暴力直接搬 checker 的 `isValid` ✓ 聪明
- 14:04 子任务 2 写成“`b[i-1] > b[i]` 就输出 1” ✗

14:01 那版暴力为什么 0 分：

```
int t = i;
wolves.clear();
while (t != 0) {
    wolves.push_back(t % 2);
    t /= 2;
}
```

`wolves` 的长度是 i 的二进制位数，不是 n 。`isValid` 第一句就检查长度，长度不够直接 false。结果只有最高位是 1（第 n 个人是狼）的分布被试过，“全是村民”（ $i = 0$ ）也从没试过。

- 题面样例三组答案：001、-1、10101——恰好最后一个都是狼或者无解，所以样例过了
- 大样例 sample2 就是子任务 1 的 1000 组数据，一跑就错 277 组。你下载了，没跑

改法：for (int k = 0; k < n; k++) wolves.push_back (i >> k & 1);。改完就是 10 分。

14:09 那版为什么 CE：solve 里调用了 solve2，但 solve2 写在 solve 下面，没声明。桌面上 T4 可执行文件是 14:00 编译的，T4.cpp 是 14:09 改的——交之前没编译。而且就算编译过了，if (n > 20) solve2 (n); 后面没有 return，还会接着跑 2^n 暴力。

子任务 2（全是 L，15 分）怎么做：左看的村民 i 要求“前面恰好 b_i 个狼”。从左往右，每个人两种状态：

- i 是村民：前面的狼数必须等于 b_i
- i 是狼：i-1 必须是村民，前面的狼数 = $b_{i-1} + 1$

每个状态的狼数都是确定的， $O(n)$ DP。你 12:52 说的“第一个人肯定说 0”就是这个 DP 的第一步。

正解（供了解）：

1. 如果知道狼的总数 W，右看的村民也能换成“前面有 $W - b_i$ 个狼”，就是上面那个两状态 DP， $O(n)$
2. 看相邻两个村民 $i \rightarrow j$ ($j = i+1$ 或 $i+2$)：
 - 都左看： $b[j] = b[i] + \text{中间狼数}$ ，和 W 无关
 - 都右看： $b[j] = b[i] - \text{中间狼数}$ ，和 W 无关
 - 一左一右：只有一个 W 能走（左→右： $W = b[i] + b[j] + \text{中间狼数}$ ；右→左： $W = b[i] + b[j] - \text{中间狼数}$ ）
3. 所以一共只有 $O(n)$ 条“挑 W 的边”。把每个位置“能不能从前一个、前两个村民走过来”写成 2×2 的 0/1 矩阵，线段树维护乘积；枚举 W 时只把这个 W 的边打开、看能不能走到终点、再关上。 $O(n \log n)$
4. 找到 W 后普通 DP 回溯出方案； $n = 1$ 时一个人当狼总是合法

大样例全部用 checker 规则校验合法；和穷举暴力随机对拍 10500 组（包括判 -1）全对； $\sum n = 2 \times 10^6$ 最慢 0.43 秒。代码 正解代码/D_狼人.cpp。

下次识别信号：有一个“全局未知量”（这里是总狼数）→ 看它在哪些地方被钉死；如果只在少数地方被钉死，就枚举它、每次只改那几处。

五、你赛时的五个“我觉得”，逐个判

你说的	判	理由
“记录所有出现过的位置太多了” (A, 10:59)	✗	状态是“第几位 + 进位”， $2n$ 个。数位 DP 从来不记整个数
“字典树退化成链，复杂度差得远” (B, 11:50)	✗	插入总代价 = $\sum s \leq 10^6$ 。链再长也是串的长度
“ 2×10^5 和 2×10^6 有啥区别” (C, 12:05)	有区别	前者允许 $O(n \log^2 n)$ ，后者必须找结构
“我认为我的思路没有问题” (A, 13:07)	✗，但你下一步做对了	反例 11011。你 10 分钟后去写暴力，这正是执行卡“我觉得我是对的 → 跑数据证明”的做法
“第二题十分，性价比不高” (14:04)	✗	B 暴力 10 分钟 10 分；你去写的 D 子任务 2 思路不对，还 CE

六、和 Day1 比：执行卡做到了哪些

规则	这场	证据
必须 freopen	✓	A、C 都拿分了
工具 5 分钟修不好就回终端	✓	12:31 Code::Blocks 抽风，直接改用终端；环境损失从 30 分钟降到 12 分钟
先看 checker	✓	12:08 读了 D 的 checker，后来暴力直接搬了 isValid
“我觉得我是对的”→ 跑数据证明	✓（晚了一点）	13:07 说完，13:17 写暴力、13:22 找到反例
用暴力对拍	✓（晚了 2 小时）	对拍一开，2 分钟就找到反例
T+0:15 四题都读完	✗	B 11:39、C 11:52、D 12:07 才读（T+1:57）
T+1:30 至少两题暴力交上	✗	第一次提交 T+2:35
每题有预算，到点换题	✗	A 用了 65%
先交暴力，再想正解	✗	C 的 30 分 12:05 想到、13:51 才交；B 的 10 分没写
最后 30 分钟：编译无警告、大样例再跑一遍	✗	D 没跑大样例、最后一交没编译

结论：环境类、工具类的规则基本做到了；时间类、先拿分类的规则一条都没做到。下场重点是后者。

七、止损点

时刻	信号	应该做	实际
10:48	大样例不对，而且不知道为什么	停，先写暴力（哪怕是 BFS 搜最少项数）	继续空想 50 分钟
11:07	写出了 $a = P - N$	5 分钟写“枚举 N”的暴力，打表	13:17 才写
T+1:00 (11:10)	还没交过任何题	停下来，挑一道写暴力交上	继续想 A
12:05	自己说“30 分手到擒来”	立刻写 C 的 $O(n^2)$ ，10 分钟	推迟到 13:36
12:28	输出 $729 \neq 671$	不交，去写暴力	12:45、13:06 各交一次错的
14:04	离结束 6 分钟，D 的新想法没把握	不再改 D；或者只修暴力的枚举	写了 solve2，没编译就交

八、经验教训

错因档案里的复发

#	病因	这是第几次	这场的表现
1	读题漏条件	2	C 写成“严格大于”（样例抓住了，没丢分）
2	复杂度盲区	2	B 以为字典树退化成链会慢，没按总和算
3	不止损、碰运气	2	A 已知 $729 \neq 671$ ，改系数试 853，还交了两次错的
4	不按子任务拿分	2	A 子任务 1、B 子任务 1、D 子任务 1/2 的暴力都没及时写
5	时间分配失控	2	A 占 65%（Day1 是 55%）
6	找理由代替验证	2	“不会是编译器的问题”“我认为我的思路没有问题”——但这次第二句之后去验证了
8	环境 / 工具问题	2	约 12 分钟（Day1 约 30 分钟），在变好

新病因

#	病因	这场的表现
9	交的不是测过的那份	A 在 5 个地方有 5 份；C 写进了 T2.cpp、B 写进了 T3.cpp；12:45 交了 12:22 的旧文件（RE）；13:06 交了已知过不了样例的；14:09 交了没编译的（CE）
10	只拿小样例验证	D 暴力过了题面样例就交，大样例（1000 组）没跑；A 对拍时 gen 还是空模板，“1000 组全对”是假的

题型规则（写进执行卡）

1. 二进制 + 加减 + 最少项数 → 数位 DP，状态记进位
2. 写出等式 → 枚举一边就是暴力
3. 估复杂度按“总和”：字典树插入总代价 = $\sum |s|$
4. 对每个前缀都要答案 → 增量维护，只改变了的项
5. 枚举 n 位的 0/1 分布：`mask >> k & 1`，k 从 0 到 n-1

调试规则

1. 对拍前 `cat 1.txt` 看一眼输入不是空的
2. 题面样例过了不算，大样例全跑一遍
3. 已知过不了样例的代码不交

比赛策略

1. T+1:00 还没交过任何题 → 立刻停，挑一道写暴力交上
 2. 自己说出“这 X 分手到擒来” → 立刻去拿，不许推迟
 3. 最后 10 分钟：只交刚编译过、跑过大样例的代码
-

九、行动清单

现在补（按顺序，约 3 小时）：

专项	练习	预计
数位 DP 带进位	不看代码重写 A 的数位 DP；做 LeetCode 2571	40 分钟
字典树 + 增量维护	洛谷 P8306 练手；然后从空白写 B	1 小时
小状态 DP	写 D 子任务 2（全是 L）的两状态 DP，用 D 的 checker 验证	40 分钟
写暴力	四题的暴力各默写一遍（正解代码/暴力/），每题限时 10 分钟	40 分钟

下次比赛：按新版《赛前执行卡》执行，重点是“零、开赛前”和“五、交之前”两节。桌面换成新版 bag（tools/vm/bag_v2/），它会帮你做“测的就是交的”和“交之前检查”。

赛后复盘时带上：归档 ZIP + AI 上下文 TXT、录音转写、虚拟机桌面导出。这次题目列表页快照里就有你的提交记录，以后归档报“本人提交 0”也不用慌。

附一：正解代码与验证记录

题	文件	做法	验证
A	正解代码/A_乘法.cpp	数位 DP， $O(n)$	样例 2/2；12 位以内所有 a 加 13~16 位随机，共 5295 个，和暴力一致；你最后 AC 的代码也全部一致
B	正解代码/B_毛线.cpp	字典树 + $mx[d]$ 增量， $O(\sum s)$	样例 5/5（最大 0.25 秒）；随机 500 组和按定义暴力一致
C	正解代码/C_棋子.cpp	见四.C， $O(n \log n)$	样例 3/3 + 题面样例；随机 900 组（ $n \leq 5000$ ，7 种分布）和 $O(n^2)$ 一致； 2×10^6 最慢约 0.4 秒
D	正解代码/D_狼人.cpp	见四.D， $O(n \log n)$	大样例全部按 checker 规则校验合法；随机 10500 组和穷举一致； $\sum n = 2 \times 10^6$ 最慢 0.43 秒
暴力	正解代码/暴力/	四题各一份，分别对应能拿的子任务分	都和正解对过

附二：赛时代码（赛时代码/，文件名里是北京时间）

文件	说明
A_v0_数1的个数_11-23.cpp	打表：数样例里 1 的个数（474）
A_v1_只试最高位借位_约10-45_抄自paper.cpp	从 paper.txt 抄出来的第一版，大样例 947
A_v2_连续1段_12-22_推断为12-45提交RE.cpp	\\ \\ 越界，本地也会段错误
A_v3_改系数_12-33.cpp	大样例 853
A_v4_13-02_13-06提交WA.cpp	大样例 729，反例 11011
A_暴力_枚举b_13-22.cpp、A_造数据_13-23.cpp	你的暴力和 gen
A_v5_加check_13-33_13-35提交AC.cpp	100 分

文件	说明
C_v1_写在T2里_严格大于_13-40.cpp	样例 1 输出 1
C_v2_写在debug里_13-49_13-51提交30分.cpp	30 分
D_v1_暴力_约14-00_14-01提交0分_还原.cpp	由 14:09 的源码去掉 solve2 还原
D_v2_加solve2_14-09_14-09提交CE.cpp	CE
B_写在T3里_未完成_14-06.cpp	开头

附三：给归档脚本的两个修正建议

1. 题名解析：D 被记成“0 Compile Error”，是把题目列表里的状态单元格拼进了题名。只取标题链接里的文字就行
2. 本人提交 403 时的兜底：比赛题目列表页本身就有“递交 状态 题目 递交者 时间 内存 语言 递交时间”表，和每题最后得分。403 时从这张表里解析，就不会报“完全未提交”